

Cyclic Proofs as Lean Tactics: A Paper-Faithful Implementation of Grotenhuis-Otten Unravelling

[Author]

June 1, 2026

Abstract

We present *CyclicTactic*, a Lean 4 library that lets users write cyclic proofs as ordinary Lean tactics and translates them into kernel-checked Lean theorems via a paper-faithful implementation of the Grotenhuis-Otten Theorem 6.1 unravelling algorithm. The system records cyclic structure into a side-channel **ProofTree** during tactic execution, validates it via the size-change principle (Lee, Jones, and Ben-Amram, 2001), augments it with Grotenhuis-Otten §5–§6 per-node data (Stack/Name/Reset annotation, RelAnc/Ineq/Hyp/cov), and emits a structural-induction proof that elaborates as the user-facing declaration. Mutual cyclic systems are handled by an analogous emitter that produces **mutual def ...end** blocks with cross-theorem σ -applied calls. We compare against Cyclist on Brotherston’s first-order benchmark and against the older Sprenger-Dam structural emitter (**Unravel.translate**) we deleted in favour of the §6-faithful path. We close by identifying the next algorithmic gap—non-structural recursion as in merge-sort—and outlining two extensions (well-founded emission, Löb induction) currently under separate development.

Contents

1	Introduction	2
1.1	Cyclic proofs in proof assistants	2
1.2	Contributions	2
1.3	Outline	3
2	Background	3
2.1	Cyclic proofs	3
2.2	Cyclist and the first-order benchmark	4
2.3	Grotenhuis-Otten unravelling	4
2.4	Related work in proof assistants	5
3	System architecture	5
3.1	Surface tactics	5
3.2	Side-channel recording	6
3.3	Two-phase elaboration	6
3.4	Pipeline overview	7
4	Implementation	8
4.1	Definition 5.1 step and Lemma 5.9 unfolding	8
4.2	Per-node §6 augmentation	8
4.3	§6 emission	8
4.4	Sort metadata and dependent binders	9

4.5	Canonical-form replacement	9
4.6	Mutual cyclic systems	10
4.7	Module layout	10
5	Evaluation	11
5.1	Worked-example suite	11
5.2	Cyclist first-order benchmark	11
5.3	Comparison with the deleted Unravel emitter	12
5.4	Code-size and elaboration overhead	12
6	Limitations and ongoing extensions	12
6.1	What the current system cannot express	12
6.2	Tree-shape versus DAG-shape: a discussion	13
6.3	The dispatcher: structural, well-founded, and Löb emission	14
6.4	Other engineering follow-ups	16

1 Introduction

1.1 Cyclic proofs in proof assistants

Cyclic proof systems offer a natural representation for reasoning about inductive predicates: rather than packaging an explicit induction hypothesis into every recursive case, the proof tree may contain *back-edges* that point to ancestor nodes, provided a global termination criterion—typically the size-change principle of Lee, Jones, and Ben-Amram [8]—is satisfied. Sprenger and Dam [10] established the equivalence between cyclic proofs and ordinary inductive proofs in the μ -calculus, formally justifying the soundness of cyclic-proof systems for first-order predicates. Brotherston’s PhD thesis [3] and the Cyclist prover [4] demonstrated the practical viability of cyclic proof search for sequent calculi with explicit trace conditions.

Despite this theoretical foundation, cyclic proofs have not seen widespread adoption in mainstream interactive proof assistants. The gap is partly ergonomic: cyclic proofs are usually presented in a sequent-calculus surface syntax with explicit bud-companion edges, which sits awkwardly inside the tactic-mode workflows that dominate Lean, Coq, and Isabelle. It is also partly methodological: a cyclic proof’s soundness depends on a non-local termination condition (size-change), and integrating this with the proof assistant’s kernel-level type-checking is a nontrivial engineering question.

A separate strand of theoretical work has recently appeared in Grotenhuis and Otten’s *Unravelling Abstract Cyclic Proofs into Proofs by Induction* [5]. They give an algorithm, applicable to any sufficiently abstract cyclic system, that translates a cyclic proof into an inductive proof by following the local annotation data they call the *Stack/Name/Reset* representation (§5) and the *relativised ancestor / inequality / hypothesis / cover* per-node data (§6). Their Theorem 6.1 then justifies a direct structural-induction emission based on this data. Crucially, their algorithm operates on abstract cyclic representations, so the translation is independent of the specific proof-system or term-language details.

1.2 Contributions

This thesis presents *CyclicTactic*, a Lean 4 library that lets users write cyclic proofs as ordinary Lean tactics and, by implementing the Grotenhuis-Otten Theorem 6.1 unravelling, replaces the user’s interactive cyclic-style proof with a kernel-checked structural-induction theorem.

Concretely, the contributions are:

1. **Side-channel tactic-mode integration.** Three primitive tactics—`cyclic <label>`, `cyc_cases x with`, and `back <label> { $\sigma$ }`—let the user write cyclic proofs inside Lean’s

standard tactic mode. Real Lean goals are visible at every cursor position; cyclic structure is recorded into a side-channel `ProofTree` value via source-position attribution (§3).

2. **A paper-faithful implementation of Grotenhuis-Otten §5–§6.** We implement the `Stack/Name/Reset` annotation step of Definition 5.1, the Lemma 5.9 key-bounded fixpoint unfolding, the Theorem 5.2 progressing-name verification, and the §6 per-node augmentation (`RelAnc`, `Ineq`, `Hyp`, `cov`, σ) needed to drive emission. To our knowledge this is the first implementation of the Grotenhuis-Otten algorithm in any proof assistant.
3. **Theorem 6.1 emission with canonical-form replacement.** The §6-driven emitter (`Theorem6.Emit.translate`) produces a Lean tactic script. After the user’s cyclic proof elaborates as a recursive `def` (giving real `InfoView` goals during writing), the environment is rolled back and the §6-emitted theorem is re-elaborated as the canonical user-facing declaration.
4. **Mutual cyclic systems.** A `cyclic_mutual` surface command, together with an event demultiplexer and a mutual emitter (`Theorem6.Emit.translateMutual`), handles cyclic proofs that span multiple mutually-recursive theorems with cross-companion back-edges. Cross-theorem buds emit as direct sibling-theorem calls, relying on Lean’s mutual-recursion termination check for soundness.
5. **Comparison with Cyclist on Brotherston’s first-order benchmark.** On the cases that both systems handle, our emitter produces proofs that match the structure Cyclist would emit. On Brotherston’s tests 10 and 11 (associativity and commutativity of `ADD`, where Cyclist fails because its first-order proof search cannot perform equational reasoning), we succeed by decoupling the cyclic structure (validated by the size-change check) from the per-arm reasoning (closed by Lean’s tactic ecosystem).

1.3 Outline

§2 reviews cyclic proof theory, the size-change principle, and the Grotenhuis-Otten papers. §3 describes the side-channel architecture and the three primitive tactics. §4 walks through the implementation of the §5 annotation and §6 augmentation+emission. §5 reports the empirical comparison against Cyclist and presents worked examples. §6 identifies the boundary cases the current system does not handle—most prominently merge-sort, whose recursion is not structural—and outlines two extensions currently under separate development: well-founded emission via Lee 2009-style measure synthesis [7], and Löb-induction emission via a step-indexed embedding [1].

2 Background

2.1 Cyclic proofs

Informally, a *cyclic proof* of a sequent $\Gamma \vdash \varphi$ is a finite tree of inference steps in which each open branch (called a *bud*) is decorated with a back-edge to an ancestor (called a *companion*) whose sequent matches the bud’s, modulo a substitution σ [3]. Cyclic proofs may be unsound in general—one can construct trivial cyclic “proofs” of false sequents—so a global termination criterion is required.

The standard criterion is Brotherston’s *trace condition*: every infinite path through the cyclic proof’s underlying graph must contain infinitely many *progressing* occurrences, where progression is defined by a fixed structural-subterm relation. Lee, Jones, and Ben-Amram’s *size-change principle* (SCT, [8]) gives an algorithmic incarnation of this idea: each back-edge is annotated with a *size-change graph* (SCG) recording which argument positions are strictly

smaller and which are non-strictly smaller; the system passes SCT iff the closure of all SCG compositions contains, in every idempotent product, at least one strict self-loop.

Sprenger and Dam’s Theorem 5 [10] shows that any cyclic proof satisfying the trace condition can be unfolded into a tree-shaped proof using nested structural induction. This is the theoretical basis for the *structural* translation strategy that all mainstream cyclic-to-inductive emitters (including Cyclist and our system) implement.

2.2 Cyclist and the first-order benchmark

Cyclist [4] is a standalone tool for cyclic sequent-calculus proof search and first-order inductive reasoning. It implements the trace condition directly and produces a separate non-interactive proof artifact. Its `benchmarks/fo/` suite contains tests 01–15, covering parity reasoning (*Od/Ev* inductive predicates), addition properties, and various ordinals.

Cyclist’s strengths are pure first-order proof search and explicit trace-condition validation; its weaknesses are equational reasoning (it cannot apply rewrite rules and so fails on the algebraic content of tests 10 and 11) and integration with mainstream proof assistants. Our system inverts this trade-off: we lose Cyclist’s automatic proof search but gain Lean’s tactic ecosystem for closing leaves and intermediate steps.

2.3 Grotenhuis-Otten unravelling

The Grotenhuis-Otten paper [5] provides the algorithmic core of our system. We summarise §5 and §6, which are the sections we implement.

Abstract cyclic systems. The paper works in an abstract setting (§2): a cyclic proof is a tree whose nodes are labelled with sequents and whose inference rules are taken from a specified set, with each bud assigned a companion ancestor. The *induced inductive system* (§3) is the target of unravelling. The authors give a general idea and case studies in §4 before introducing the key constructions in §5–§6.

§5: Annotated cyclic representations. The paper defines (Definition 5.1) a per-node annotation consisting of:

- a Stack function assigning each argument position a stack of fresh *names*,
- a Name set tracking which names are currently alive, and
- a Reset function recording which names have been “reset” by the most recent step.

The Definition 5.1 *step* takes a parent annotation and an edge SCG and produces the child annotation by (i) allocating fresh names per child slot, (ii) reusing ancestor stacks where the SCG permits, (iii) running a reset loop to fixpoint to capture the per-name dependency structure.

Lemma 5.4 bounds the alphabet at $2^m + m$ names where m is the arity, so by a Königs-lemma argument every cyclic proof has a finite annotation reached after finitely many unfolding steps. Lemma 5.9 gives the algorithmic shape of this unfolding: when a bud’s initial annotation fails to satisfy Theorem 5.2’s progressing-name condition, iterate the annotation up to a key-bounded fixpoint.

Theorem 5.2: Existence of a cyclic representation satisfying name preservation. This theorem says the §5 annotation can be chosen so that every bud has a *progressing name*: a name in $\text{Reset}(\text{bud}) \cap \text{preserved names}$ along the cycle. Verifying Theorem 5.2 amounts to checking, for each bud, that such a name exists.

§6: Relativised ancestor data and Theorem 6.1. Building on the §5 annotation, the paper defines per-node:

- $\text{RelAnc}(n_k)$, the relativised-ancestor set;
- $\text{Ineq}(n_k)$, the inequalities the path enforces;
- $\text{Hyp}(n_k)$, the induction hypotheses in scope (one per *sprout* ancestor that introduced a fresh IH);
- $\text{cov}(b)$ for each bud, the variable that “covers” the progressing name (i.e., binds the IH the bud applies);
- σ (Lemma 6.4), the substitution mapping sprout slot variables to bud slot variables and the progressing slot variable to cov .

Theorem 6.1 then states that the cyclic proof unfolds to a structural induction proof in which:

- every *sprout* (a node that introduces a fresh IH in Hyp) emits as **induction** `<var> generalizing <rest> with`;
- every other internal node emits as the corresponding C-rule (**cases** for case-splits, etc.);
- every bud emits as an IH application with arguments determined by σ .

The paper continues with applications (§7), a multi-sort extension (§8), and appendices on cyclic Heyting arithmetic (App. B) and multi-sorted inductive systems (App. C). We implement §5 and §6 in full but do not address §7–§8 or the appendices.

2.4 Related work in proof assistants

To our knowledge, no mainstream proof assistant has previously shipped a paper-faithful implementation of cyclic-to-inductive unravelling. The Iris framework [6] provides cyclic-style reasoning via the $\triangleright$ modality and Löb induction, but it sits at a different point in the design space: Iris proofs live in a step-indexed embedding rather than being translated into ordinary inductive proofs. The connection between Iris-style Löb reasoning and cyclic proofs is a known correspondence [9, 2] and forms the basis for the Löb-induction extension under separate development (§6).

Wehr’s PhD thesis [11] surveys the cyclic proof theory landscape comprehensively and gives a fresh treatment of SCT-derived induction orders in §3.2 (Theorem 3.2.4), which our system uses as an auxiliary check alongside the Grotenhuis-Otten annotation.

The older Sprenger-Dam translation tradition is implemented in Cyclist and in our pre-§6 baseline (`Unravel.translate`, since deleted). The present work is the first to embed the Grotenhuis-Otten §6 algorithm itself in a proof assistant.

3 System architecture

This section describes the surface tactics, the side-channel architecture, and the two-phase elaboration flow that lets users write cyclic proofs interactively while producing a kernel-checked structural-induction theorem as the user-facing declaration.

3.1 Surface tactics

CyclicTactic provides three primitive tactics, plus two command-level forms.

cyclic <label>. Registers the current goal as a named *companion*. Subsequent back-edges within the same theorem body refer to this companion by label. Internally, **cyclic** pushes a `.companion lbl seq` event into the recorder (described below) and, for the first **cyclic** call in a body, captures the goal head constant for use as `defaultSimpPred` when the emitter needs to unfold the goal predicate.

cyc_cases x with | <pat> => Case-splits on `x` while recording arm boundaries. Each arm’s syntactic range is captured so the recorder can later attribute back-edges to the correct arm. The tactic delegates the actual case-split to Lean’s built-in **cases**, so InfoView goals appear as usual; the only additional work is the event push.

back <label> [{ σ }]. Closes the current goal via a recursive call to the enclosing theorem. The σ substitution specifies how each binder of the enclosing theorem maps to the call’s arguments. **back R {n := n', m := m}** desugars to **exact <name> n' m**, which Lean’s standard structural-recursion check accepts when σ produces strictly smaller arguments. The back-edge’s source position, σ , and surrounding source text are pushed as a `.back` event.

Command-level forms. `cyclic_thm name (binders) : type by tactics` packages the three primitives into a command that produces a single user-facing declaration: the kernel-checked structural-induction theorem produced by §6 emission, or the fallback recursive form if §6 emission fails. `cyclic_mutual ...end_mutual` extends the same flow to mutually-recursive blocks (§4.6).

3.2 Side-channel recording

The cyclic structure of a proof—which case-splits, which back-edges, which σ s—is not visible to Lean’s standard tactic machinery. Rather than modifying Lean to track this information, `CyclicTactic` uses a *side-channel* architecture: each of the three primitive tactics pushes an event into a global `CyclicState IO.Ref`; events are then post-processed into a `ProofTree` value (a strongly-typed cyclic-proof representation defined in `CyclicTactic/ProofTree.lean`).

The event-recording approach has three benefits over modifying Lean: (1) the recorder is independent of Lean’s elaborator internals, so it survives Lean version upgrades; (2) the user gets ordinary InfoView goals during proof writing, with no special editor support needed; (3) the recorded structure is a pure-data Lean term, which we can validate, analyse, and transform offline.

```
inductive CyclicEvent where
| companion (label : String) (sequent : Sequent)
| caseSplitStart (var : String) (sequent : Sequent) (arms : List ArmInfo)
| caseSplitEnd
| back (ancestor : String) (sequent : Sequent)  $\sigma$ ( : Subst)
  (pos : Nat) (sourceText : String)
```

The `ArmInfo` entries carry syntactic positions for each arm. The `Build.eventsToTree` function (in `CyclicTactic/Build.lean`) walks the linear event stream: each `caseSplitStart/caseSplitEnd` scope is converted to a `ProofTree.caseSplit`; back-edges within an arm are mapped to that arm by source-position attribution.

3.3 Two-phase elaboration

The most subtle architectural choice is that `cyclic_thm` elaborates the proof *twice* in succession: first as a recursive **def**—which gives InfoView goals during interactive writing—then, if §6 emission succeeds, as a structural-induction **theorem** that becomes the user-facing declaration. The two phases never coexist in the environment.

```

-- Phase A: snapshot env before any work
let envBefore ← Lean.getEnv

-- Phase B: elaborate recursive form. User's tactics fire here;
-- InfoView shows real Lean goals at every cursor position.
Lean.Elab.Command.elabCommand←
  ( `(def $name $binders : $type := by $tacs))

-- Phase C: snapshot env-with-recursive as fallback
let envWithRecursive ← Lean.getEnv

-- Phase D: build ProofTree from the recorded events;
-- run SCT + induction-order + $5 + $6 validations.
...

-- Phase E: try canonical $6 emission
let script := Theorem6.Emit.translate ...
match parse(script) with
| .ok cmdStx =>
  Lean.setEnv envBefore -- roll back
  try elabCommand cmdStx -- canonical form
  if hasGoodValue then () -- $6 emission committed
  else Lean.setEnv envWithRecursive -- restore fallback
| .error _ =>
  -- recursive form stays as the user-facing decl

```

The user-facing `<name>` is always exactly one declaration. When §6 emission succeeds it is the structural-induction theorem; when §6 emission fails for any reason it is the recursive `def` produced by Phase B.

This architecture is what enables the system to be both ergonomic and paper-faithful: the user writes cyclic-style with real interactivity, but the kernel-checked artifact is the structural-induction proof Theorem 6.1 unravels. To our knowledge no prior cyclic-proof system in a mainstream proof assistant uses this two-phase pattern.

3.4 Pipeline overview

The end-to-end flow from `cyclic_thm` source to kernel-checked theorem proceeds through nine stages, listed below with their owning modules. `CyclicTactic/PIPELINE.md` walks the same flow on a multi-recursive example (`btPredT`).

1. **Command parsing and Phase A env snapshot** (`Tactic.lean::elabCyclicThm`).
2. **Phase B recursive-form elaboration** (Lean's standard `def` machinery).
3. **Event recording** (three custom tactics push into `CyclicState`).
4. **ProofTree construction** (`Build.lean::eventsToTree`, via source-position attribution).
5. **SCT validation** (`ProofTree.extractTraceSCGsLabeled` + `SizeChange.checkMultiSCT`).
6. **Paper-faithful §5 annotation** (`PaperAnnotation.lean`: Definition 5.1 step + Lemma 5.9 unfolding + Theorem 5.2 verification).
7. **§6 augmentation** (`Theorem6.lean::computeAug`: `RelAnc/Ineq/Hyp/cov/σ` per node).
8. **§6 emission** (`Theorem6.Emit.translate`).
9. **Canonical replacement** (Phase E rollback and re-elaboration).

4 Implementation

This section describes the implementation of the §5 annotation, the §6 augmentation, the §6 emission for single-theorem proofs, and the extension to mutual cyclic systems. Throughout we cite specific files and (where relevant) function names in the repository.

4.1 Definition 5.1 step and Lemma 5.9 unfolding

The §5 annotation lives in `CyclicTactic/PaperAnnotation.lean`. We implement:

- `step` : `NodeAnnot` $\rightarrow$ `SCGraph` $\rightarrow$ `NodeAnnot`, the Definition 5.1 step.
- `freshNames` : `Nat` $\rightarrow$ `List Name` $\rightarrow$ `List Name` for the smallest-available naming convention.
- `resetLoop` for the reset-to-fixpoint operation.
- `annotateTree` : `ProofTree` $\rightarrow$ `List BudAnnot`, which walks a proof tree and produces the per-bud annotation.
- `unfoldFixpoint`, which iterates `annotateTree` on a single failing bud up to a key-bounded fuel (16 iterations by default, well above the Lemma 5.4 bound for the proof sizes we have seen).
- `checkTheorem52`, which verifies the progressing-name condition.

The implementation closely follows the paper’s prose. The two non-obvious details are the smallest-available naming convention (needed for the Lemma 5.4 alphabet bound to hold structurally) and the key-based history tracking inside `unfoldFixpoint` (so the fixpoint is detected when a bud’s annotation *key* recurs, not just its value).

4.2 Per-node §6 augmentation

The §6 per-node data lives in `CyclicTactic/Theorem6.lean`. The top-level entry point is:

```
def computeAug (t : ProofTree) : NodeAugMap
```

returning, for each node label ℓ , an `NodeAug` record containing `RelAnc`, `Ineq`, `Hyp`, and (for bud nodes) the `bud?` info: the bud label, its progressing name, its cover, and the progressing slot index.

The walker `computeAugAux` threads a current annotation `cur` : `NodeAnnot` and a path substitution `$\sigma p$` : `Subst` down the tree. At each step it computes the parent’s effective slots (the `rootArgs` under `$\sigma p$`), the child’s effective slots (the `rootArgs` under the child’s `$\sigma c$` , except at back-edges where the child’s literal sequent is used), and the edge SCG (a strict comparison between the two slot lists, slot-by-slot).

The asymmetric treatment of back-edges versus other children is load-bearing: at a back-edge, the bud’s sequent already reflects the user’s σ substitution from the recursive call, so comparing it against the parent’s path-narrowed slots reveals strict descent. This asymmetry, encoded in `effectiveSlots`, is what lets the §6 augmentation detect the recursion structure the user wrote.

4.3 §6 emission

The emitter `Theorem6.Emit.translate` walks the augmented tree and produces a Lean tactic-script string. The structural choice per node is:

- A `caseSplit` node *at a sprout* (i.e., a node where Hyp extends the parent's) emits as `induction <var> generalizing <rest> with`, where `<rest>` is the `rootArgs` except for the case-split variable.
- A `caseSplit` node *not at a sprout* emits as `cases <var> with` (Lemma 6.3 C-rule selection).
- A back node emits as `exact ih_<recVar> <args>`, where `<recVar>` is read from σ at the prog slot and `<args>` are the σ -substituted rest-of-slots.
- A node "branch" (multi-recursive arm) emits as `simp [<pred>]; refine ⟨?, ...⟩; u child; ...` so that each branch's bud emits its IH call with the correct cov-determined IH name.
- Leaves emit the user-supplied closing tactic (if any) or a default `simp`.

The decision about `induction` versus `cases` is the most important fidelity gain over the pre-§6 emitter we deleted (`Unravel.translate`, which always emitted `induction`). In proofs where an inner case-split's subtree contains no bud that uses its fresh IH, the §6 emitter correctly emits `cases`, avoiding an unused `ih_` binder.

The top-level function:

```
def translate (defaultSimpPred : Option String) (goalType thmName : String)
  (varSorts : List (String & SortInfo)) (t : ProofTree) : String
```

calls `computeAug` `t` to build the per-node augmentation, then walks the tree via `emitTree`, then assembles the result into a `theorem <name> <binders> : <goalType> := by <body>` string ready to hand to `Lean.Parser.runParserCategory`.

4.4 Sort metadata and dependent binders

The emitter needs to know each binder's inductive type to emit the right constructor headers (e.g., `succ n` for `Nat`, `node l r` for binary trees). This information is stored as `SortInfo` records (`CyclicTactic/EmitCommon.lean`) and built by `Build.buildSortInfo` from a `Lean Expr`.

For binders whose types are non-dependent, the elaboration is straightforward (`Lean.Elab.Term.elabType` on the binder's type syntax). For *dependent* binders like `(h : Od n)` where `n` is an earlier binder, the per-binder `elabType` call would fail because `n` is not in scope. We resolve this by reading binder types from the *already-elaborated recursive def's signature* (via `Lean.Meta.forallTelescope`), which has the dependent context properly built. This handling is in `Tactic.lean` immediately after the §6 augmentation is computed.

4.5 Canonical-form replacement

The Phase E replacement step is implemented as:

```
match Lean.Parser.runParserCategory envWithRecursive 'command script with
| .ok cmdStx =>
  Lean.setEnv envBefore -- roll back
  let beforeMsgs := ←( getThe ...).messages
  try Lean.Elab.Command.elabCommand cmdStx
  catch _ => pure ()
  let envAfter ←Lean.getEnv
  let hasGoodValue : Bool :=
    match envAfter.find? name.getId with
    | some di => match di.value? with
      | some v => !v.hasSorry
      | none => true
    | none => false
```

```
if hasGoodValue then () -- §6 form committed
else Lean.setEnv envWithRecursive -- restore fallback
```

We check `hasSorry` rather than checking whether elaboration threw an exception, because Lean’s elaborator may emit error *diagnostics* (turning the value into one containing `sorry`) without raising; we treat any sorry-containing value as a §6 emission failure.

4.6 Mutual cyclic systems

The `cyclic_mutual` command extends the same flow to mutually-recursive blocks. The implementation surfaces are:

Event demultiplexing. Inside a `cyclic_mutual` block, events from all entries are interleaved in the single event stream of `CyclicState`. The function `demuxMutualEvents` (in `Tactic.lean`) walks the chronological stream, using the pre-registered *companion-target table* (each `cyclic <label>` declaration’s label $\mapsto$ owning theorem) to attribute each event to a specific theorem. The result is a list of per-entry event subsets.

Per-entry tree construction. For each entry’s events, `Build.eventsToTree` produces a per-entry `ProofTree`. Each tree’s back-edges reference companion labels that may belong to *other* entries (cross-companion edges).

Mutual emitter. `Theorem6.Emit.translateMutual` emits a `mutual def ...end` block. Cross-theorem buds (whose `ancestor` resolves to a sibling entry’s companion via the companion-target table) emit as `exact <sibling-thm-name> < $\sigma$ -applied args>`, relying on Lean’s mutual-recursion termination check for soundness. Within-entry case-splits are emitted as `cases`, since the cyclic structure in the mutual case is provided by Lean’s mutual-recursion check, not by within-tree sprouts.

§6 augmentation skipped for mutual. The §6 augmentation (`computeAug`) is skipped for mutual blocks because `annotateTree`’s Lemma 5.9 unfolding does not terminate when a bud’s ancestor is not in the tree’s ancestor stack (the cross-theorem case). Since cross-theorem buds emit as direct sibling calls—not IH applications—the §6 augmentation is unused in this case anyway. We return to this limitation in §6: `cyclic_mutual` blocks with within-entry cycles are not yet supported.

4.7 Module layout

The implementation totals approximately 5 400 lines of Lean code across the following modules:

Module	LOC
SizeChange.lean	171
ProofTree.lean	400
Extract.lean	125
Measure.lean	285
Annotation.lean	288
InductionOrder.lean	307
Reorganize.lean	369
PaperAnnotation.lean	811
Theorem6.lean	755
EmitCommon.lean	155
Tactic.lean	897
Build.lean	372
Design.lean	448
Total	5 383

5 Evaluation

We evaluate `CyclicTactic` on three axes: (1) the worked-example suite under `CyclicTactic/Examples/`; (2) Brotherston’s first-order benchmark `benchmarks/fo/` from the Cyclist distribution [4]; and (3) qualitative comparison with the older Sprenger-Dam-style structural emitter we deleted in favour of the §6 path.

5.1 Worked-example suite

The repository contains 18 `cyclic_thm` and `cyclic_mutual` declarations distributed across five example files. All 18 elaborate via the canonical §6 path (the `[cyclic_thm <name>] canonical` form: `Theorem 6.1 emission ✓` diagnostic fires for each). The breakdown is:

File	Count	Coverage
Smoke.lean	3	basic Nat induction, single back-edge
Probe.lean	5	multi-arg, branches, $\exists$ -witness, <code>have</code>
Probe2.lean	5	enums, lists, nested back-edge
drp.lean	4	comparison with the Cyclic DSL baseline
MutualSmoke.lean	1	mutual block (Od/Ev)

The examples exercise every shape the system handles: single-recursive case splits (e.g., `zeroAddT`, `addCommT`), multi-recursive branches (`btPredT`), nested case-splits with lex descent (`ackTotal_cyc`), existential witnesses (`existsEqT`), intermediate `have` steps (`zeroAddHaveT`), and cross-companion mutual blocks (`od_is_nlike_cyc` / `ev_is_nlike_cyc`).

5.2 Cyclist first-order benchmark

Cyclist’s `benchmarks/fo/` suite contains 15 numbered tests. We ported the cases that map cleanly to Lean: tests 01 (mutual parity), 07 (commutative addition), 09 (`ADD` successor), 10 (associativity), and 11 (commutativity). The results are summarised in Table 1.

Tests 10 and 11 are the headline win. Cyclist fails because its first-order proof search applies sequent rules without equational simplification, so it cannot manipulate intermediate `ADD`-predicate hypotheses algebraically. Our system, hosted on Lean, decouples the cyclic structure (validated by SCT and §6 augmentation) from the per-arm reasoning (closed by Lean tactics), so `congrArg Nat.succ` plus `rw` discharges the algebraic step inside each arm.

Table 1: Cyclist comparison on `benchmarks/fo/` tests we ported.

#	Sequent (cyclist notation)	Cyclist	CyclicTactic
01	$O(x) \vdash N(x)$ (mutual)	✓	✓ (<code>cyclic_mutual</code>)
07	$N(x) \vdash ADD(x, 0, x)$	✓	✓
09	$ADD(x, y, z) \vdash ADD(x, sy, sz)$	✓	✓
10	associativity of ADD	✗	✓
11	commutativity of ADD	✗	✓

Tests 02, 04, 12, 13, 14, 15 are not ported in the current system. Tests 02 and 04 require introducing an induction hypothesis as a hypothesis and case-splitting on its disjunctive result—our `back` primitive consumes the goal directly via `exact`, so this pattern is not expressible within the current tactic surface. Tests 12, 13, 15 require well-founded emission with an explicit measure (Sprenger-Dam two-step descent, R/R2 lex, 2-Hydra)—our current §6 emitter is structural-only.

5.3 Comparison with the deleted Unravel emitter

The pre-§6 baseline (`Unravel.translate`, deleted as part of the §6 work) was a Sprenger-Dam-flavoured structural emitter that always emitted `induction var with` for every case-split. The §6 emitter, by contrast, distinguishes sprouts (case-splits whose subtree contains a bud that uses the freshly-introduced IH) from non-sprouts (case-splits whose subtree’s buds use IHs from an enclosing sprout). At a non-sprout it emits `cases var with` instead, dropping the unused `ih_` binder.

For 12 of the 17 single `cyclic_thm` examples, both emitters produced character-identical scripts. For the remaining 5 (`isColorT`, `fZeroT`, `maxNonneg`, `ackTotal_cyc` inner split, `existsEqT`, and `ackTotalT`), the §6 emitter chose `cases` where the older emitter chose `induction`. In each case the §6 choice is the paper-faithful one (Lemma 6.3’s rule selection) and produces an equally-elaborable proof with one fewer unused binder.

5.4 Code-size and elaboration overhead

The §6 emission adds approximately 1.5 seconds of elaboration time per `cyclic_thm` on a modern laptop (this is dominated by the second `Lean.Elab.Command.elabCommand` invocation in Phase E). Total build time of the worked-example suite (excluding the `CyclistComparison.lean` file, which hangs on a pre-existing `SubjectTerm.toString` stack overflow unrelated to this work) is under 30 seconds for all 18 examples combined.

The emitted proof terms are typically within a factor of 1.5 of the hand-written structural-induction proof’s term size, as measured by Lean’s `#print` output. We have not yet measured this systematically across the example suite.

6 Limitations and ongoing extensions

6.1 What the current system cannot express

The current §6-structural emitter handles cyclic proofs whose recursion fits Lean’s structural-recursion check. Several classes of proofs lie outside this scope.

Non-structural recursion. The most visible limitation is exposed by merge-sort. The auxiliary lemma `length (merge xs ys) = length xs + length ys` recurses on the result of a Decidable comparison: when $x \leq y$, it descends on `xs`; otherwise on `ys`. Neither argument is structurally smaller across both cases, so Lean’s structural-recursion check rejects the underlying

def produced by Phase B, and the structural §6 emitter cannot witness termination on any single variable. This is precisely the case the dispatcher (§6.3) detects and routes to the well-founded backend, where a lexicographic measure (`xs.length, ys.length`) takes over; merge-sort additionally exposes a residual arithmetic-discharge gap (the +1 step the hand proof leaves to `omega`) that is independent of the dispatch.

Decidable case-splits. The original `cyc_cases` tactic recorded only inductive-type case splits, so a proof body containing `by_cases h : P` with back-edges in each branch was opaque to the event recorder. The `cyc_by_cases` tactic (`CyclicTactic/Tactic.lean`) now closes this gap: it records a Decidable split as a `.dCaseSplitStart` event the same way `cyc_cases` records inductive ones, so the back-edges within each branch enter the size-change check and the proof can dispatch to WF emission (§6.3).

DAG-shaped cyclic proofs. The system assumes back-edges target ancestors only. Cyclic proofs that share intermediate sub-derivations across multiple branches (DAG-shaped proofs in the sense of Brotherston’s PhD [3]) cannot be expressed directly. The standard workaround—factoring shared sub-derivations into separate `cyclic_thms` or `cyclic_mutual` entries—works for most practical proofs but is not always idiomatic. We discuss the theoretical equivalence in §6.2 below.

Multi-companion in a single theorem. The §6 data model assumes `companion = root`: every back-edge in a single `cyclic_thm` body cycles to the same companion (the root sequent). Proofs whose natural shape uses multiple `cyclic` declarations within one theorem body (with different buds targeting different companions) are not expressible. `cyclic_mutual` offers a workaround by splitting into mutually-recursive entries, but the surface ergonomics differ.

Within-entry cycles in `cyclic_mutual`. The mutual emitter currently skips §6 augmentation, because `annotateTree`’s Lemma 5.9 unfolding does not terminate on cross-theorem buds. A `cyclic_mutual` block whose entries contain within-entry back-edges (in addition to cross-companion edges) is therefore not supported. The §6 framework’s extension to mutual systems is, to our knowledge, an open theoretical question.

Proposition 5.8 verification. Our system uses Wehr 3.2.4’s lex-order finder (Theorem 3.2.4 of [11]) as a check on the induction order, complementary to the Grotenhuis-Otten Theorem 5.2 progressing-name verification. The paper’s Proposition 5.8 gives a stronger verification of the induction order that we do not implement; we delegate this to Lean’s structural-recursion check. Implementing Proposition 5.8 directly would turn the current diagnostic PASS/FAIL into a real proof obligation.

Multi-sort extension (§8). Grotenhuis-Otten §8 extends the unravelling algorithm to multi-sorted inductive systems. We currently handle single-sorted cyclic proofs; extending to §8 would let cyclic proofs span fundamentally distinct inductive types as their recursion structure.

6.2 Tree-shape versus DAG-shape: a discussion

It is worth being precise about what is lost when we restrict to tree-shaped cyclic proofs with back-edges to ancestors only. Sprenger and Dam’s Theorem 5 and its descendants establish that tree-shaped and DAG-shaped cyclic proofs prove the same sequents: any DAG can be unfolded to a tree by duplicating shared sub-derivations, with potentially exponential blow-up. So provability is not affected.

What *is* affected:

- **Proof size.** A DAG with many shared sub-derivations unfolds to a tree exponentially larger.
- **Sharing-natural patterns.** Symmetry-driven proofs, cross-recursive lemmas with shared support, and confluence-style arguments naturally have DAG shape.
- **Lateral cycles.** A back-edge from one branch of a proof tree to a node in another branch (not an ancestor) cannot be represented.

For the Lean tactic-mode setting we target, this restriction is less constraining than it sounds. Lean idiom favours factoring shared sub-derivations into separate **theorems** or local **have** binders—both of which absorb the sharing-naturalness patterns above. The remaining genuinely lateral case (a single proof tree with a non-ancestor back-edge) is rare in practice and can typically be expressed by promoting the lateral target to a separate **cyclic_thm**.

This trade-off is most visible against Cyclist’s separation-logic benchmark (**benchmarks/sl/**), where DAG shape is heavily used by the proof-search procedure and our system would struggle. For Cyclist’s first-order benchmark and for typical Lean proof work, tree-shape is a defensible architectural choice rather than a deficiency.

6.3 The dispatcher: structural, well-founded, and Löb emission

The Grotenhuis-Otten §6 algorithm assumes structural emission is always possible. It is not: merge-sort and the swap-recursion of Cyclist’s Test 14 (§6) admit no single-variable structural order. The system therefore *dispatches* among emission strategies on the basis of the size-change data, rather than committing to structural emission unconditionally. Two of the three strategies are now implemented on the **wf** branch; the third remains under separate development. We describe each, then state the characterisation theorem they converge on.

The dispatch predicate. The choice between structural and well-founded emission is made by a single Boolean predicate, **canStructural** (**CyclicTactic/Tactic.lean**):

```
def canStructural (tree : ProofTree)
  (labeled : List (String E SCGraph)) (arity : Nat) : Bool :=
  match InductionOrder.findInductionOrder tree labeled arity with
  | none => false
  | some asg => (InductionOrder.lexOrderFromAssignment asg).length <= 1
```

findInductionOrder runs the Wehr 3.2.4 lex-order finder over the bud-companion strongly-connected components; it returns an assignment of progressing argument positions, or **none** when no consistent order exists. **canStructural** returns **true** exactly when that assignment, after deduplication, uses *at most one* argument position—i.e. when a single **induction** on one variable carries the whole recursion, which is precisely the shape Theorem 6.1’s structural emitter (§4.3) produces. When the order needs two or more positions (a genuine lexicographic descent), or does not exist at all, the predicate returns **false** and the proof is routed to the well-founded backend. Merge-sort’s **merge** fails the predicate because its $x \leq y$ arm descends on **xs** (position 0) and its $x > y$ arm on **ys** (position 1): no single position is preserved-and-progressing across both arms, so **findInductionOrder** returns **none**.

Well-founded (WF) emission. When **canStructural** is **false**, the system emits **def ... termination_by <measure>** with a **decreasing_by simp_wf**; **omega** discharge, via **WFEmit.translate** (**CyclicTactic/WFEmit.lean**). The measure is synthesised from the per-back-edge size-change graphs by **synthMeasure** (**CyclicTactic/Measure.lean**), which tries four schemas in increasing generality (the lex-ranking ideas are Lee 2009’s [7]):

1. **Lex over a permutation** of all argument positions—a strict self-loop at some position, nonstrict at all earlier ones (classic lex termination, e.g. Ackermann).
2. **Lex over an ordered subset**, allowing non-participating positions to be dropped from the tuple.
3. **Sum measure** $a_0 + \dots + a_{n-1}$, for swap-style recursions where the bag of arguments shrinks as a whole.
4. **Closure-witness greedy lex**: compute the SCT composition closure, take each idempotent’s strict-self-loop positions as a soundness witness, and greedily build a lex order covering the most uncovered idempotents at each step. This is a flat-arity specialisation of the Grotenhuis-Otten Definition 5.1 reset-proof construction.

Every schema validates its candidate against the *input* graphs, not merely the closure idempotents, because Lean’s `termination_by` requires every recursive call to decrease, not just every cycle. The result is a `Measure`, which is either `lex_order` or `sum_arity`. The Decidable case-splits that such proofs require (merge-sort’s $x \leq y$) are now recorded by the `cyc_by_cases` tactic, which emits a `.dCaseSplitStart` event exactly as `cyc_cases` does for inductive splits; this closes the gap noted in §6 above. The swap recursion of Test 14 ($N2(x, y)$ with recursive call $N2(y, x)$) goes through end-to-end: `synthMeasure` produces `sum 2`, rendered `termination_by x + y`, and the emitted `n2_holds` is a kernel-checked term (`CyclicTactic/Examples/WFSuite.lean`).

Löb-induction emission. A theoretically stronger alternative replaces structural and WF emission entirely with *Löb induction* via a step-indexed embedding [1, 6]. Every SCT-validated cyclic proof can be emitted as a Löb application, using a $\triangleright$ modality defined by $\triangleright P n := \forall m < n, P m$ and the Löb axiom $(\forall n. \triangleright P n \rightarrow P n) \rightarrow \forall n. P n$ proved from `Nat.strongRec`. The advantage is universality: Löb emission applies to any SCT-valid cyclic proof, with no case analysis on which strategy fits. The disadvantage is artefact form: emitted proofs are about $\triangleright P$ rather than P directly, which is less idiomatic for Lean users. This extension is under active development on a separate `lob` branch in a sibling repository, and is the only one of the three not yet integrated into the main dispatcher.

The characterisation theorem. The dispatcher `canStructural` and the synthesiser `synthMeasure` are total, decidable functions on the §6 size-change data; what is *conjectural* is that their decisions agree with Lean’s own acceptance of the emitted artefact. We separate the two.

Conjecture 6.1 (Dispatch soundness). *Let T be an SCT-valid `ProofTree` with labelled size-change graphs G and arity n , and write $A = \text{computeAug } T$ for its §6 augmentation.*

1. *If $\text{canStructural } T G n = \text{true}$, then the structural emitter `Emit.translate` T produces a term Lean’s structural-recursion check accepts.*
2. *If $\text{canStructural } T G n = \text{false}$ and $\text{synthMeasure } G n = \text{some } M$, then the well-founded emitter `WFEmit.translate` $M T$ produces a `def` whose `termination_by` M obligation *decreasing_by* discharges.*
3. *The residual case $\text{canStructural} = \text{false}$ and $\text{synthMeasure} = \text{none}$ —an SCT-valid proof whose descent is captured by no lex-or-sum measure—is exactly the regime in which the first two strategies fail and Löb emission is required.*

Conjecture 6.2 (Löb universality). *For every SCT-valid `ProofTree` T , the Löb emitter `Emit.translateLob` T produces a Lean-acceptable term.*

The two extensions thus carve up the SCT-valid proofs into a structural core (Conjecture 6.1.1), a well-founded middle (Conjecture 6.1.2), and a Löb-only remainder (Conjectures 6.1.3 and 6.2). Clauses 1 and 2 are now backed by working emitters and an example suite; proving that the *syntactic* predicates `canStructural` and `synthMeasure` soundly approximate Lean’s *semantic* acceptance—and that the Löb remainder is genuinely universal—are the load-bearing claims that anchor the next stage of the work. A proof of Conjecture 6.1.1 would proceed by showing the Wehr 3.2.4 single-position assignment is exactly the witness Lean’s structural-recursion elaborator searches for; of clause 2, by appeal to the soundness of the Lee 2009 ranking schemas against the input graphs; and of Conjecture 6.2, by exhibiting the step-indexed model in which $\triangleright$ is the strict-order approximation and reading SCT validity as the descent that makes the Löb premise discharge.

6.4 Other engineering follow-ups

Several smaller items are not algorithmically interesting but are worth listing:

- The `CyclicTactic/Examples/CyclistComparison.lean` file has a pre-existing stack overflow in `SubjectTerm.toString`, triggered by one of the later examples in the file. The crash predates the §6 work and likely involves infinite term traversal when an inductive-predicate hypothesis is recorded. Investigating this is independent of the §6 / WF / Löb work but blocks bare `lake build`.
- The system does not yet support `cyclic_thm` bodies whose structure is *not* a sequence of `cyc_cases` + `back` + leaves. In particular, intermediate `cyclic` re-declarations within a single body are not handled, and interleaving `cyclic_thm`-style tactics with other custom tactics from libraries is untested.
- Diagnostic output is verbose but ad hoc; a structured `[cyclic_thm]` info schema (so external tooling can parse the dispatcher’s decisions) would make integration with editor extensions easier.
- Performance of the §6 augmentation has not been profiled. For very deep proof trees the `annotateTree` + `unfoldFixpoint` cost could become significant; we have not observed this on the example suite.

References

- [1] Robert Atkey and Conor McBride. Productive coprogramming with guarded recursion. In *ICFP*, 2013.
- [2] Stefano Berardi and Makoto Tatsuta. Intuitionistic podelski-rybalchenko theorem and equivalence between inductive definitions and cyclic proofs, 2017. arXiv:1712.03502.
- [3] James Brotherston. *Sequent Calculus Proof Systems for Inductive Definitions*. PhD thesis, University of Edinburgh, 2006.
- [4] James Brotherston, Nikos Gorogiannis, and Rasmus Lerchedahl Petersen. A generic cyclic theorem prover. In *APLAS*, 2012.
- [5] Lide Grotenhuis and Daniël Otten. Unravelling abstract cyclic proofs into proofs by induction, 2026. arXiv:2602.12054.
- [6] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. In *POPL*, 2015.

- [7] Chin Soon Lee. Ranking functions for size-change termination. *ACM Trans. Program. Lang. Syst.*, 31(3), 2009.
- [8] Chin Soon Lee, Neil D. Jones, and Amir M. Ben-Amram. The size-change principle for program termination. In *POPL*, 2001.
- [9] Alex Simpson. Cyclic arithmetic is equivalent to Peano arithmetic. In *FoSSaCS*, 2017.
- [10] Christoph Sprenger and Mads Dam. On the structure of inductive reasoning: Circular and tree-shaped proofs in the μ -calculus. In *FoSSaCS*, 2003.
- [11] Dominik Wehr. *Cyclic Proof Theory*. PhD thesis, [University], 2025.